



SISTEMA DE GESTÃO DE FALHAS INDUSTRIAIS

DISCIPLINA: ENGENHARIA DE SOFTWARE

Estratégia de Versionamento

Controle de Versão com Git e GitHub

Autores:

Anna Lívia Vasconcellos, Gabriel Dias,
Lucas Sanches Marcilio, Mirella Costa
Viana e Thalyson Gama da Costa

Funções:

Versionamento e Integração

v1.0.3 - 12 de junho de 2026

Resumo

Este documento descreve a estratégia de versionamento adotada pelo grupo no desenvolvimento do Sistema de Gestão de Falhas de Manutenção Industrial, utilizando **Git** como sistema de controle de versão e o **GitHub** como plataforma de hospedagem e colaboração. São apresentados: o link do repositório, a estratégia de *branches* adotada (*feature branches* por padrão de projeto), a convenção de mensagens de *commit*, as evidências de *merge* e integração via *Pull Requests* e o histórico de *commits* do grupo, demonstrando a evolução incremental e colaborativa do projeto.

Índice

1	Repositório do Projeto	4
2	Estratégia de Branches Adotada	4
2.1	Modelo escolhido: Feature Branch Workflow	4
2.2	Branches do repositório	4
2.3	Fluxo de trabalho	5
3	Padrão de Commits Utilizado	6
3.1	Convenção adotada	6
3.2	Regras seguidas pelo grupo	6
4	Evidências de Merge e Integração	6
4.1	Pull Requests	6
4.2	Commits de merge	6
5	Histórico de Commits do Grupo	8
6	Conclusão	9

1 Repositório do Projeto

O código-fonte, os testes, a documentação e os diagramas do projeto estão versionados em um repositório público no GitHub:

Link do repositório

https://github.com/LucasSanchesM/Projeto02_Bruno

O repositório contém a implementação dos protótipos em Java, os testes unitários com JUnit e stubs, a documentação técnica (JavaDoc), os diagramas de classes (arquivos Astah e imagens) e os documentos do projeto.

2 Estratégia de Branches Adotada

2.1 Modelo escolhido: Feature Branch Workflow

O grupo adotou o modelo **Feature Branch Workflow**: a branch **master** é a linha principal e estável do projeto, e cada nova funcionalidade — em especial, a aplicação de cada padrão de projeto GoF e a criação dos respectivos stubs e testes — foi desenvolvida em uma *branch* dedicada, integrada à **master** somente após revisão.

Por que Feature Branch Workflow?

O modelo foi escolhido por ser simples o suficiente para um projeto acadêmico com cinco integrantes e, ao mesmo tempo, garantir isolamento: cada padrão de projeto (State, Strategy, Observer, Factory Method e Facade) evoluiu em sua própria branch, sem quebrar a linha principal. A integração via *Pull Request* permitiu revisão pelos colegas antes do *merge*, e os conflitos foram resolvidos de forma controlada.

2.2 Branches do repositório

A Tabela 1 relaciona as branches criadas durante o projeto e suas finalidades.

Branch	Finalidade
master	Linha principal e estável; recebe as integrações via merge/PR.
state-falha	Implementação do padrão State para o ciclo de vida da Falha .
Strategy	Estruturação inicial do padrão Strategy de priorização.
feature/strategy-pattern	Evolução do padrão Strategy com nomenclatura de feature branch.
StrategyStubs	Stubs e testes unitários das estratégias de priorização.
observer	Implementação do padrão Observer (monitor de SLA, notificações e painel).
AplicandoStubObserver	Stubs e correções dos testes do padrão Observer.
FactoryMethod	Implementação do padrão Factory Method para criação de funcionários.
ImplementaçãoStubFactory	Stub e reorganização de pacotes para os testes do Factory.
Facade-testes	Testes do padrão Facade e refatoração do fluxo de gestão de falhas.
Facade-Stub	Criação do stub do Facade e refatoração dos testes.
Docs	Atualizações de documentação (documento arquitetural, casos de teste, diagramas).

Tabela 1: Branches do repositório e respectivas finalidades.

2.3 Fluxo de trabalho

O fluxo seguido pelos integrantes foi:

1. Criar uma branch a partir da **master** para a funcionalidade (ex.: `git checkout -b state-falha`);
2. Desenvolver com commits pequenos e incrementais;
3. Publicar a branch no GitHub (`git push origin <branch>`);
4. Abrir um *Pull Request* para a **master**;
5. Revisão por outro integrante e resolução de eventuais conflitos;
6. *Merge* na **master** e continuidade do ciclo.

Comunicação do grupo

Além do fluxo de trabalho no GitHub, o grupo realizou **chamadas de áudio semanais** para alinhamento do desenvolvimento. Nessas reuniões, os integrantes tiravam dúvidas sobre os padrões de projeto, revisavam juntos os *Pull Requests* abertos, combinavam a divisão das próximas tarefas e se ajudavam mutuamente na resolução de problemas de código e de conflitos de *merge*, garantindo que todos acompanhassem a evolução do projeto.

3 Padrão de Commits Utilizado

3.1 Convenção adotada

O grupo adotou mensagens de commit **descritivas em português**, sempre no formato de uma frase curta que resume *o que* foi feito, inspirando-se gradualmente na convenção *Conventional Commits* (prefixos `feat:`, `docs:`, etc.) nos commits mais recentes:

```
<tipo opcional>: <descricao curta no imperativo/passado, em portugues>

Exemplos reais do repositório:
feat: atualizando o documento com a imagem atualizada do diagrama do EstadoFalha
docs: atualiza casos de teste e documento de arquitetura
Implementado o stub para o teste do factory
Criando o Stub Facade
Resolvido o conflito de branchs no stub de falha do projeto
```

Listagem 1: Estrutura adotada para mensagens de commit.

3.2 Regras seguidas pelo grupo

- **Commits pequenos e incrementais:** cada commit representa uma etapa lógica da evolução (criação de stub, refatoração, correção, documentação);
- **Mensagem descritiva:** a mensagem indica claramente o artefato afetado (padrão, stub, teste ou documento);
- **Prefixos semânticos** (`feat:`, `docs:`) nos commits de funcionalidade e documentação mais recentes;
- **Um assunto por commit:** alterações de código, testes e documentação foram, em geral, separadas em commits distintos.

4 Evidências de Merge e Integração

4.1 Pull Requests

A integração das branches à `master` foi realizada majoritariamente por meio de *Pull Requests* (PRs) no GitHub, conforme a Tabela 2.

PR	Branch de origem	Título	Situação
#1	state-falha	State falha	Mesclado
#2	Facade-testes	Facade testes	Mesclado
#3	Docs	Docs	Fechado
#4	ImplementaçãoStubFactory	Implementação stub factory	Mesclado
#5	Facade-Stub	Facade stub	Mesclado

Tabela 2: Pull Requests abertos no repositório.

4.2 Commits de merge

O histórico da `master` registra os commits de *merge* que evidenciam a integração contínua das branches, incluindo a resolução explícita de conflito:

```
b75ab8b Merge pull request #5 from LucasSanchesM/Facade-Stub
6316441 Merge pull request #4 from LucasSanchesM/ImplementacaoStubFactory
d79c112 Merge pull request #2 from LucasSanchesM/Facade-testes
814ff60 Merge pull request #1 from LucasSanchesM/state-falha
038b17e Merge branch 'AplicandoStubObserver'
c718079 Resolvido o conflito de branches no stub de falha do projeto
```

Listagem 2: Commits de merge registrados no historico do repositório.

Resolução de conflitos

Durante a integração da branch `AplicandoStubObserver`, ocorreu conflito entre as alterações do stub de `Falha`. O conflito foi resolvido manualmente e registrado no commit `c718079` (“Resolvido o conflito de branches no stub de falha do projeto”), demonstrando o uso correto do fluxo de resolução de conflitos do Git.

5 Histórico de Commits do Grupo

A listagem a seguir apresenta um recorte do histórico de commits do repositório (do mais recente para o mais antigo), evidenciando a evolução incremental do projeto:

d791ae4	Gabriel Dias	12/06	corrigindo versao do documento arquitetura
d099046	Gabriel Dias	12/06	docs: atualiza casos de teste e documento de arquitetura
9db194c	Gabriel Dias	12/06	adicionando pasta que contem os arquivos asta dos diagramas
1eba029	Lucas Sanches	12/06	Corrigido alguns detalhes na documentacao do codigo
27cf9ea	Lucas Sanches	12/06	Corrigido ciclo de vida de falha do facade
0934ee4	Thalyson	11/06	Terminada documentacao do padrao Strategy e seus stubs
c135571	Lucas Sanches	11/06	Feita a padronizacao de nomes de classes
c718079	Lucas Sanches	11/06	Resolvido o conflito de branchs no stub de falha
038b17e	Lucas Sanches	11/06	Merge branch 'AplicandoStubObserver'
09aaabb	Ana Livia	11/06	Correcao de erros no ObserverStub e no ObserverTest
b75ab8b	Lucas Sanches	11/06	Merge pull request #5 (Facade-Stub)
b1dcd9a	Ana Livia	11/06	Implementacao da Stub no metodo observer
6c02edd	Mirella	11/06	Documentacao Facade Stubs Atualizada
9333a7e	Mirella	11/06	Documentacao Facade Test Atualizada
5d197e6	Mirella	11/06	Documentacao Src Facade atualizada
400f172	Thalyson	11/06	Ajuste no Stub e teste de priorizacao por categoria
16440dc	Mirella	11/06	Subindo os testes refatorados
028ae9b	Mirella	11/06	Criando o Stub Facade
c6d3bac	Mirella	11/06	Implementacao do stub nos testes de metodos individuais
27d1314	Mirella	11/06	Refatoracao do Facade para o funcionamento correto do fluxo
c5af338	Mirella	10/06	Subindo o construtor com injecao
6997b4d	Thalyson	10/06	Incluindo a utilizacao inicial de stubs em Strategy
d77bf27	Lucas Sanches	09/06	Transferido interface de estado falha para pacote services
6316441	Lucas Sanches	09/06	Merge pull request #4 (ImplementacaoStubFactory)
50a6628	Thalyson	04/06	Atualizando e impondo as regras do padrao Strategy
5357897	Thalyson	01/06	Estruturacao inicial padrao Strategy
b8b79b7	Lucas Sanches	09/06	feito a padronizacao dos testes
e33ad8a	Lucas Sanches	09/06	Implementado o stub para o teste do factory
79606eb	Lucas Sanches	09/06	Iniciacao da separacao de classes em model e services
553ad2e	Gabriel Dias	09/06	feat: atualizando o documento com a imagem do EstadoFalha

Listagem 3: Recorte do historico de commits do repositorio.

O histórico completo pode ser consultado diretamente no repositório: https://github.com/LucasSanchesM/Projeto02_Bruno/commits/master

Evolução incremental

O histórico evidencia a evolução incremental do projeto: primeiro a estruturação dos padrões (State, Strategy, Factory), em seguida a refatoração do Facade, depois a introdução dos stubs nos testes de cada padrão e, por fim, os ajustes de documentação e padronização de nomes — cada etapa registrada em commits pequenos e rastreáveis.

6 Conclusão

A estratégia de versionamento adotada — *Feature Branch Workflow* com integração via *Pull Requests* — mostrou-se adequada ao contexto do projeto. As branches por padrão de projeto isolaram o desenvolvimento de cada funcionalidade, os commits incrementais e descritivos tornaram a evolução do sistema rastreável, e os *merges* registrados (incluindo a resolução explícita de conflito) evidenciam a integração contínua do trabalho, atendendo integralmente aos requisitos do entregável de Estratégia de Versionamento.